

# Decisiones técnicas del proyecto Poll.inc

Este documento resume las decisiones técnicas tomadas en el desarrollo de la plataforma de encuestas **Poll.inc**, incluyendo tecnologías principales, patrón de layout, organización de componentes, gestión de datos, formularios y visualización de resultados.

## Tecnologías principales

- **Framework:** `Next.js 16` (App Router).
  - **Librería base:** `React 19`.
  - **Lenguaje:** TypeScript.
  - **Estilos:** `Tailwind CSS 4` + algunas utilidades en `globals.css`.
  - **Gestión de datos remotos:** `@tanstack/react-query`.
  - **HTTP client:** `axios`.
  - **Formularios:** `Formik`.
  - **Validación:** `Yup`.
  - **Gráficos:** `chart.js` + `react-chartjs-2`.
- 

## 1. Arquitectura general y patrón App Shell

La aplicación está construida con **Next.js 16 (App Router)** y **React 19**, utilizando el archivo `app/layout.tsx` como punto central de configuración.

Se adoptó el **patrón App Shell**, que consiste en definir una “cáscara” de aplicación que se mantiene fija mientras cambia solo el contenido de cada página.

En este proyecto:

- `app/layout.tsx` define:
  - El árbol raíz de React (`<html>`, `<body>`).
  - Estilos globales y tipografía.
  - El montaje del proveedor global de React Query.
  - El componente `AppShell`, que actúa como layout base.
- `AppShell` envuelve a todas las páginas (`children`) y es responsable de la estructura visual común:
  - Distribución general.
  - Fondo base de la aplicación.
  - Header y footer globales.
- Las páginas concretas:

- `app/page.tsx` → Home (lista de encuestas y botón “Crear encuesta”).
- `app/polls/[id]/page.tsx` → Pantalla para responder una encuesta.
- `app/polls/[id]/results/page.tsx` → Pantalla para visualizar los resultados.

Todas estas páginas se renderizan **dentro** del App Shell.

## Justificación del App Shell

- **Consistencia visual**

Todas las vistas comparten el mismo fondo, tipografía y estilo base, al estar envueltas por el mismo `AppShell`.

- **Mantenibilidad**

Cualquier cambio global de layout o estética (por ejemplo, colores base, estructura del contenedor) se realiza en un solo lugar (`layout.tsx` / `AppShell`), sin tener que modificar cada página.

- **Lógica global centralizada**

El `ReactQueryProvider` se monta a nivel de App Shell, por lo que todos los Client Components disponen del contexto de React Query sin configuración duplicada.

- **Preparado para crecer**

Si se decide agregar navegación global, footer, selector de tema o información del trabajo práctico, se incorpora en `AppShell` y aparece automáticamente en todas las pantallas.

## 2. Estilos y diseño (Tailwind CSS)

Se eligió **Tailwind CSS 4** como sistema principal de estilos por:

- Su enfoque de **utility-first**, que acelera la construcción de interfaces.
- La facilidad para mantener un diseño consistente en todas las vistas.
- La reducción de CSS global complejo.

Uso concreto:

- `app/globals.css`:
  - Importa Tailwind.

- Define variables de color, fuentes y algunas clases de layout como `layout-container` y `layout-content-container`.
- Componentes y páginas:
  - Utilizan clases Tailwind para estructura (`flex`, `grid`, `min-h-screen`, `max-w-2xl` / `max-w-3xl`).
  - Utilizan una paleta basada en tonos teal/verde y grises oscuros: `bg-[#3F5F5F]`, `border-[#4F6F6F]`, `text-off-white`, `text-desaturated-teal`.
  - Los elementos clave (cards, botones, encabezados) se construyen con combinaciones de `rounded-xl`, `shadow-lg`, `hover:bg-...`, etc.

Esto permite que la home, la pantalla de votación y la de resultados mantengan el mismo estilo general y se perciban como parte de un mismo producto.

### 3. Server Components vs Client Components

Se definió una separación clara entre:

- **Server Components:**

Páginas y layout que no necesitan hooks de React y se benefician de ejecución en el servidor:

- `app/layout.tsx`
- `app/page.tsx`
- `app/polls/[id]/page.tsx`
- `app/polls/[id]/results/page.tsx`

- **Client Components:**

Componentes que necesitan:

- Hooks de React (`useState`, `useRouter`, etc.).
- Hooks de React Query (`useQuery`, `useMutation`).
- Formularios interactivos (Formik).
- Gráficos (Chart.js).

- Entre ellos:

- `ReactQueryProvider`
- `AppShell`
- `PollList`
- `RespondPollForm`
- `PollChart`
- Hooks de datos en `app/hooks/usePoll.ts`

Este enfoque aprovecha las ventajas de los Server Components (menos JavaScript enviado al cliente) para el layout y las páginas, y reserva los Client Components para la lógica dinámica e interactiva.

## 4. Gestión de datos con React Query

Para la comunicación con la API interna de Next y el manejo de estado remoto se utiliza [@tanstack/react-query](#).

### Hooks definidos ([app/hooks/usePoll.ts](#))

- [useGetAllPolls\(\)](#)
  - [useQuery](#) que llama a `GET /api/polls`.
  - Se usa en [PollList](#) para listar las encuestas creadas.
- [useGetPoll\(id: string\)](#)
  - [useQuery](#) que llama a `GET /api/polls/[id]`.
  - Se usa en [RespondPollForm](#) para obtener el título y las preguntas de una encuesta.
- [useCreatePoll\(\)](#)
  - [useMutation](#) que llama a `POST /api/polls`.
  - Permite crear nuevas encuestas a partir de un formulario.
- [useSubmitResponse\(\)](#)
  - [useMutation](#) que llama a `POST /api/polls/[id]/responses`.
  - En `onSuccess` invalida el query `["pollResults", pollId]`, forzando que los resultados se actualicen cuando el usuario vote.
  - Se usa en [RespondPollForm](#).
- [useGetPollResults\(id: string\)](#)
  - [useQuery](#) que llama a `GET /api/polls/[id]/results`.
  - Se configura con `refetchInterval: 3000` para refrescar los resultados cada 3 segundos (efecto “tiempo real”).
  - Se usa en [PollChart](#).

### Justificación

- React Query simplifica:
  - Manejo de estados `loading`, `error` y `success`.
  - Cache de peticiones y reutilización de datos.
  - Revalidación automática cuando se actualizan recursos (mutations + invalidation).
- Encaja naturalmente con el patrón App Shell y los Client Components, ya que todo vive bajo un único [ReactQueryProvider](#).

## 5. Formularios y validación (Formik + Yup)

### Formularios en el cliente (Formik)

Se usa **Formik** en `RespondPollForm` para manejar el formulario de respuesta a una encuesta:

- Los `initialValues` se generan dinámicamente:
  - Un campo por pregunta: `{ [questionId]: optionIdSeleccionada }`.
- Cada pregunta se renderiza como un conjunto de opciones `radio`:
  - `name` = id de la pregunta.
  - `value` = id de la opción.
- Antes de enviar:
  - Se verifica que todas las preguntas tengan respuesta.
  - Si falta alguna, se muestra un mensaje y no se hace el submit.
  - Si todo es válido, se llama a `useSubmitResponse` y, en caso de éxito, se redirige a `/polls/[id]/results`.

### Validación en la API (Yup)

Se utiliza **Yup** para validar los datos de creación de encuestas del lado del servidor:

- Endpoint: `POST /api/polls` (`app/api/polls/route.ts`).
- Esquema: `pollCreationSchema`.
- Configuración de Yup:
  - `abortEarly: false` → se reportan todos los errores de validación.
  - `stripUnknown: true` → se descartan campos no esperados.

Si la validación falla, la API devuelve:

- `status 400` con `error: "Datos inválidos"` y un listado de errores.

Si la validación pasa:

- Se asignan IDs únicos (`crypto.randomUUID()`) a cada pregunta y opción.
- Se persiste la encuesta a través de la clase `Database`.

## 6. Visualización de resultados (Chart.js + react-chartjs-2)

Para mostrar los resultados de cada encuesta se usa:

- **chart.js** como motor de gráficos.
- **react-chartjs-2** como wrapper para integrarlo con React.

### Implementación en PollChart

- Se registran los módulos necesarios de Chart.js:
  - **CategoryScale**, **LinearScale**, **BarElement**, **ArcElement**, **Title**, **Tooltip**, **Legend**.
- Se definen dos tipos de gráfico:
  - **Histograma (Bar)**: votos por opción en el eje Y.
  - **Gráfico de torta (Pie)**: distribución porcentual de votos por opción.
- El usuario puede alternar entre histogramas y tortas mediante un botón.
- Se utiliza una paleta de colores fija para las barras/segmentos, con bordes blancos para mejorar el contraste.
- Se ajustan los **defaults** de Chart.js (color y tamaño de fuentes) y las opciones de **ticks**, **tooltip** y **legend** para que los textos sean legibles sobre fondos oscuros.
- Además del gráfico, se muestra un resumen textual:
  - Cantidad de votos por opción.
  - Porcentaje de cada opción dentro de la pregunta.

### Justificación

- Chart.js es suficiente para el nivel de visualización requerido en el trabajo práctico.
- Con **react-chartjs-2** la integración en un Client Component es directa.
- La combinación permite un dashboard de resultados claro y con buena legibilidad en el contexto visual de la app.

## 7. Persistencia de datos para el TP

Dado que el objetivo es un trabajo práctico y no un producto en producción, se simplificó la persistencia utilizando un archivo JSON.

- Archivo: `database.json` en la raíz del proyecto.
- Acceso encapsulado en `app/lib/database.ts`, con la clase `Database`.
- Estructura principal:
  - `polls: Poll[]`
  - `responses: PollResponse[]`

Métodos relevantes:

- `getAllPolls()` → lista las encuestas.
- `createPoll(poll)` → crea una nueva encuesta con IDs generados.
- `getPollById(id)` → devuelve una encuesta específica.
- `addResponse(response)` → agrega una respuesta a una encuesta.
- `getResponsesByPollId(pollId)` → obtiene respuestas para calcular resultados.
- `hasVoted(pollId, voterIdentifier)` → permite detectar votaciones repetidas desde el mismo identificador (IP).

Esta solución es suficiente para simular una base de datos en un entorno local y facilita la corrección del trabajo práctico sin necesidad de desplegar infraestructura adicional.

## Conclusión

La arquitectura de **Poll.inc** combina:

- Un **patrón App Shell** que define la estructura global de la interfaz y centraliza los providers comunes.
- Una separación clara entre **Server Components** (layout y páginas) y **Client Components** (formularios, hooks de datos, gráficos).
- Gestión de datos robusta con React Query.
- Formularios dinámicos validados con Formik + Yup.
- Visualización de resultados con Chart.js adaptada a un diseño de fondo oscuro.
- Una persistencia sencilla basada en `database.json` adecuada para los objetivos del trabajo práctico.